Design & Analysis of Algorithms

Unit 1 · Exam notes in easy language · M23DE0303 (MCA Sem III, REVA University) · From Dr. Lakshmi K's slides; results marked "slides show only the code" are standard textbook results added to complete the topic.

> **The whole unit in 4 lines**
> An **algorithm** is a step-by-step recipe to solve a problem. We then ask: **how fast is it, and how does it slow down when the input gets big?** We answer that by **counting the main operation**, and describing the count with **O, Ω, Θ**. For loops we add up the count (summation). For recursion we write a **recurrence** and solve it.

1. What is an algorithm?

An **algorithm** is a sequence of clear instructions that takes any valid input and gives the required output in a **finite amount of time**. It is an idea, so it does not depend on any programming language.

5 characteristics (very common question)

Property	Easy meaning
Input	Zero or more values are given from outside.
Output	At least one result comes out.
Definiteness	Every step is clear, with only one meaning.
Finiteness	It must stop after a limited number of steps, for every input.
Efficiency	Every step is simple and quick to do.

Example: GCD of two numbers

Euclid's method. Idea: gcd(m, n) = gcd(n, m mod n). Keep replacing the pair until the second number is 0.

```
while n ≠ 0 do
    r ← m mod n
    m ← n
    n ← r
return m
```

Trace for gcd(60, 24): 60 mod 24 = 12 → pair becomes (24, 12) → 24 mod 12 = 0 → pair becomes (12, 0) → answer **12**.

Consecutive integer checking (the second method in the slides): set t = min(m, n). If t divides both m and n, return t. Otherwise reduce t by 1 and repeat. It works, but it is much slower than Euclid. *Lesson: one problem can have many algorithms with very different speeds.*

2. Steps of designing and analysing an algorithm

1. **Understand the problem**: what are the objects, and what operations apply to them?
2. **Decide the computational means**: what can the machine do?

3. **Exact or approximate solution?** Some problems can only be solved approximately.
4. **Choose data structures.**
5. **Choose a design technique**: a general approach such as Brute Force, Divide and Conquer, Dynamic Programming, Greedy.
6. **Design the algorithm** and write it as natural language, pseudocode or a flowchart (3 ways).
7. **Prove correctness**: it must give the right output for every valid input, in finite time. Use a correct formula and induction. For approximate algorithms, the error must stay within a given limit.
8. **Analyse it**: time efficiency (how fast), space efficiency (how much extra memory), simplicity, generality (range of inputs) and optimality (no other algorithm can do better).
9. **Code it** in C, C++ or Java, and make it efficient (move unchanging expressions out of loops, reuse common sub-expressions, replace costly operations with cheap ones).

3. Important problem types

Type	Easy meaning	Examples / key words
Sorting	Arrange items in ascending order (numbers, strings, or records using a key).	**In-place** = needs no extra memory (Quick sort). **Stable** = equal items keep their original order.
Searching	Find a given value (the search key) in a collection.	Linear search, binary search
String processing	Work on sequences of characters.	String matching = finding a word inside a text; DNA strings use {A, C, G, T}
Graph problems	Points (vertices) joined by lines (edges).	Traversal, shortest path, topological sort, travelling salesman, graph colouring
Combinatorial	Find a permutation, combination or subset that meets conditions (often max value or min cost). The hardest class.	Subset sum, knapsack, graph colouring
Numerical	Equations, integrals, evaluating functions. Mostly solved only approximately.	
Geometric	Points, lines, polygons.	**Closest pair** (two nearest points), **convex hull** (smallest convex polygon covering all points)

Fundamental data structures

- **Linear:** array, linked list, stack, queue (operations: search, insert, delete; can be static or dynamic).
- **Non-linear:** graphs and trees. A **tree** is a connected graph with no cycles. A graph is stored as an **adjacency list** or an **adjacency matrix**.
- **Set** = unordered, distinct elements. **Bag** = unordered, repeats allowed. **Dictionary** = a bag where you can search, add and delete.

4. Maths you need (quick version)

Topic	Easy meaning
Set	A collection of distinct things, e.g. A = {2, 4, 6, 8}. Written by listing (roster) or by a rule (set-builder): {x | x is even and x < 10}. Types: finite, infinite, empty (ø), universal (U).
Subset, power set	A ⊆ B means every element of A is in B. Power set P(A) = all subsets of A. For A = {1, 2}: P(A) = {ø, {1}, {2}, {1, 2}}.
Set operations	Union A ∪ B (in A or B) · Intersection A ∩ B (in both) · Difference A − B (in A, not in B) · Complement A′ (not in A) · Cartesian product A × B (all ordered pairs).
Function	Each input goes to exactly **one** output. Domain = inputs, codomain = allowed outputs, range = outputs actually produced. Example f(x) = x^2: range is the non-negative numbers.
Types of function	**One-to-one:** no two inputs share an output. **Onto:** every codomain element is hit. **Bijective:** both, so it can be reversed. **Constant:** f(x) = k. **Identity:** f(x) = x.
Relation	A subset of A × B, e.g. R = {(1,3), (2,4)}. **Reflexive:** (a, a) always there. **Symmetric:** (a, b) gives (b, a). **Transitive:** (a, b) and (b, c) give (a, c). **Equivalence** = reflexive + symmetric + transitive. **Partial order** = reflexive + antisymmetric + transitive.
Vector	A quantity with size and direction, v = (v_1, ..., v_n). Add or subtract part by part; scalar multiplication multiplies every part. **Dot product** = a_1b_1 + ... + a_nb_n (a number). **Cross product** (3D) gives a vector perpendicular to both.
Matrix	Numbers in rows and columns. Types: row, column, square, diagonal, identity (1s on the diagonal), zero. Add or subtract only if the sizes are equal.
Linear equation / inequality	a_1x_1 + ... + a_nx_n = b (e.g. 3x + 2y = 7). An inequality uses <, >, ≤, ≥ (e.g. 3x + 4 < 10).

Where used: sets → hash sets, dictionaries, removing duplicates · functions → hash functions · relations → graph edges, database keys · vectors and matrices → neural networks, graphics, image processing, adjacency matrices.

5. Measuring how good an algorithm is

Three ideas

1. **Input size (n).** The number that says how big the input is. For sorting or searching, n = number of items. For a number n (like a primality test), size = number of bits = $\lfloor \log_2 n \rfloor + 1$.
2. **Basic operation.** Counting every step is too hard, so count only the **most important operation** (usually in the innermost loop). Running time ≈ c_{op} × C(n), where c_{op} is the time of one basic operation and C(n) is how many times it runs.
3. **Order of growth.** For small n, nothing matters. For large n, only the **shape** of the growth matters.

Problem	Input size	Basic operation
Search a key in a list	Number of items n	Comparing with the key
Multiply two matrices	Matrix order n (or number of elements)	Multiplying two numbers
Check if n is prime	Number of binary digits of n	Division

Problem	Input size	Basic operation
Graph problem	Number of vertices and/or edges	Visiting a vertex or using an edge

How fast things grow (slowest growth is best)

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

n	$\log_2 n$	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	33	100	1,000	1,000	3.6 million
100	6.6	660	10,000	1 million	1.3×10^{30}	9.3×10^{157}

Class	Name	Where it comes from
1	Constant	Best case of a few algorithms
log n	Logarithmic	Cutting the problem down by a fixed fraction every step (binary search)
n	Linear	Scanning a list once (sequential search)
n log n	Linearithmic	Divide and conquer (merge sort, quick sort on average)
n^2	Quadratic	Two nested loops, simple sorts
n^3	Cubic	Three nested loops, matrix multiplication
2^n	Exponential	Generating all subsets of n items
n!	Factorial	Generating all orderings (permutations) of n items

Best, worst and average case

Some algorithms take different time on different inputs of the **same size**.

- **Worst case:** the input that makes it run longest.
- **Best case:** the input that makes it run shortest.
- **Average case:** the expected count on a typical input. **It is NOT (best + worst) ÷ 2.**

Example: sequential search (look at items one by one until the key is found). Think of finding a name in an unsorted list.

- **Best:** key is the first item → 1 comparison.
- **Worst:** key is last, or missing → n comparisons.
- **Average:** let p = chance the key is present, all positions equally likely.

$$C_{avg}(n) = p(n + 1)/2 + n(1 - p)$$

If the key is certainly present (p = 1): (n + 1)/2, about half the list. If it is certainly absent (p = 0): n.

6. Big-O, Ω and Θ in plain words

We compare our running time **t(n)** with a simple function **g(n)**, ignoring small n and constant factors.

Notation	Think of it as	Rule (for some constants $c > 0$ and n_0, for all $n \geq n_0$)
O(g(n))	"at most" (≤) upper bound	$t(n) \leq c \cdot g(n)$
Ω(g(n))	"at least" (≥) lower bound	$t(n) \geq c \cdot g(n)$
Θ(g(n))	"exactly about" (=) tight bound	$c_1 \cdot g(n) \leq t(n) \leq c_2 \cdot g(n)$

Graph meaning: for large n, the curve t(n) stays **below** c·g(n) for O, **above** c·g(n) for Ω, and **between** c_1·g(n) and c_2·g(n) for Θ. **Little-o:** t(n) is o(g(n)) if $t(n)/g(n) \to 0$, so t grows strictly slower than g (rarely used).

How to prove (the exam recipe)

Pick c and n_0, then show the inequality. Always **write the values of c and n_0**.

Example A: $100n + 5 \in O(n^2)$
For $n \geq 5$: $100n + 5 \leq 100n + n = 101n \leq 101n^2$. So **$c = 101$, $n_0 = 5$**.
Same function in O(n): for $n \geq 1$, $100n + 5 \leq 100n + 5n = 105n$. So **$c = 105$, $n_0 = 1$**.

Example B: $n^3 + 10n^2 + 4n + 2 \in \Omega(n^2)$
For $n \geq 0$ the left side is $\geq n^2$. So **$c = 1$, $n_0 = 0$**.

Example C: $\frac{1}{2} \cdot n(n-1) \in \Theta(n^2)$
Upper: $\frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$ for all $n \geq 0$.
Lower: $\frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{4}n^2$ for all $n \geq 2$.
So **$c_2 = \frac{1}{2}$, $c_1 = \frac{1}{4}$, $n_0 = 2$**.

Sum rule (when an algorithm has two parts)

If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then

$$t_1(n) + t_2(n) \in O(\max\{ g_1(n), g_2(n) \})$$

In words: when two parts run one after the other, the **slower part decides** the answer. It also works for Ω and Θ.

Proof idea: $t_1 \leq c_1 g_1$ for $n \geq n_1$, and $t_2 \leq c_2 g_2$ for $n \geq n_2$. Let $c_3 = \max\{c_1, c_2\}$ and $n \geq \max\{n_1, n_2\}$. Then $t_1 + t_2 \leq c_3(g_1 + g_2) \leq 2c_3 \cdot \max\{g_1, g_2\}$. So the constants are $2c_3$ and $\max\{n_1, n_2\}$.

Limit shortcut

Compute the limit of t(n) / g(n) as n becomes very large:

Limit value	Meaning	Notation
0	t grows **slower** than g	$t \in O(g)$
a positive number c	t and g grow **equally fast**	$t \in \Theta(g)$
∞	t grows **faster** than g	$t \in \Omega(g)$

Helpers: **L'Hôpital's rule** (limit of t/g = limit of t′/g′) and **Stirling's formula** $n! \approx \sqrt{(2\pi n)} \cdot (n/e)^n$.

Example 1: $\frac{1}{2}n(n-1)$ vs n^2. The limit is $\frac{1}{2}$ (a positive number) → same order, $\Theta(n^2)$.
Example 2: $\log_2 n$ vs $\sqrt{n}$. Using L'Hôpital the limit is 0 → $\log n$ grows slower than $\sqrt{n}$, so $\log n \in o(\sqrt{n})$.

7. Analysing non-recursive algorithms (loops)

The 5-step recipe

1. Decide the input size n.
2. Find the basic operation (usually in the innermost loop).
3. Does the count depend only on n? If not, do worst, average and best case separately.
4. Write a **sum** for how many times the basic operation runs.
5. Simplify the sum with standard formulas to get a closed form or at least the order of growth.

Formulas to remember

$$\sum_{i=l}^{u} 1 = u - l + 1 \qquad \sum_{i=1}^{n} i = n(n+1)/2 \approx \tfrac{1}{2}n^2 \in \Theta(n^2)$$

Also: constants come out of a sum, and the sum of (a ± b) splits into two sums.

Example 1: Maximum element

```
maxval ← A[0]
for i ← 1 to n−1 do
    if A[i] > maxval  then  maxval ← A[i]
return maxval
```

Basic operation: the comparison `A[i] > maxval`. It runs once for each i from 1 to n − 1:

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

Example 2: Are all elements distinct?

```
for i ← 0 to n−2 do
    for j ← i+1 to n−1 do
        if A[i] = A[j] return false
return true
```

Basic operation: `A[i] = A[j]`. The count depends on the input, so take the **worst case** (all distinct, or the only equal pair is at the end). For each i, the inner loop runs (n − 1 − i) times:

$$C_{worst}(n) = \sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \ldots + 1 = n(n-1)/2 \approx \tfrac{1}{2}n^2 \in \Theta(n^2)$$

Example 3: Matrix multiplication (C = A × B, n × n)

```
for i ← 0 to n−1 do
    for j ← 0 to n−1 do
        C[i,j] ← 0
        for k ← 0 to n−1 do
            C[i,j] ← C[i,j] + A[i,k] × B[k,j]
```

Basic operation: the multiplication in the innermost loop. Three loops, each running n times:

$$M(n) = n \times n \times n = n^3 \in \Theta(n^3)$$

Example 4: Number of binary digits of n

```
count ← 1
while n > 1 do
    count ← count + 1
    n ← ⌊n / 2⌋
return count
```

The loop halves n each time, so it runs about $\log_2 n$ times → $\Theta(\log n)$. (The slides give only the code; this result is the standard one.)

8. Analysing recursive algorithms

A recursive function **calls itself**. We cannot just sum loops, so we write a **recurrence**: the cost for size n in terms of the cost for a smaller size.

The 5-step recipe

1. Decide the input size.
2. Find the basic operation.
3. Check whether the count changes for different inputs of the same size.
4. Write the **recurrence** and its **starting value**.
5. Solve it by **backward substitution** (keep replacing, spot the pattern) and give the order of growth.

Example 1: Factorial

```
F(n):  if n = 0 return 1
       else return F(n−1) * n
```

Basic operation: multiplication. M(n) = multiplications.

$$M(n) = M(n-1) + 1, \quad M(0) = 0$$

Substitute again and again: M(n) = M(n−1) + 1 = M(n−2) + 2 = M(n−3) + 3 = … = M(n−n) + n = **n**. So it is **Θ(n)**.

Example 2: Tower of Hanoi

Move n discs from the source pole to the destination pole using a third pole. Rules: one disc at a time, only the top disc can move, never put a bigger disc on a smaller one.

```
TowerOfHanoi(n, S, T, D):
    if n = 1: move the disc from S to D
    else:
        TowerOfHanoi(n−1, S, D, T)   // move n−1 discs out of the way
        move the largest disc from S to D
        TowerOfHanoi(n−1, T, S, D)   // bring the n−1 discs on top of it
```

Basic operation: moving one disc. Move n−1 discs, move 1, move n−1 again:

$$M(n) = 2M(n-1) + 1, \quad M(1) = 1$$

Backward substitution: $M(n) = 2M(n-1) + 1 = 2^2M(n-2) + 2 + 1 = 2^3M(n-3) + 2^2 + 2 + 1 = \ldots = 2^iM(n-i) + 2^i - 1$. Put $i = n - 1$:

$$M(n) = 2^{n-1} \cdot 1 + 2^{n-1} - 1 = \mathbf{2^n - 1} \in \Theta(2^n)$$

Check: 2 discs need 3 moves, 3 discs need 7. The call tree has 2^l calls at level l, and $2^0 + \ldots + 2^{n-1} = 2^n - 1$. **Exponential, so it becomes impossible quickly.**

Example 3: BinRec, number of binary digits

```
BinRec(n):  if n = 1 return 1
            else return BinRec(⌊n/2⌋) + 1
```

Basic operation: the addition. One addition per halving:

$$A(n) = A(\lfloor n/2 \rfloor) + 1, \quad A(1) = 0$$

The floor $\lfloor\ \rfloor$ makes substitution messy, so first take $n = 2^k$. This is allowed by the **smoothness rule**: *the order of growth found for $n = 2^k$ is also correct for all other n* (under broad conditions).

$A(2^k) = A(2^{k-1}) + 1 = A(2^{k-2}) + 2 = \ldots = A(2^0) + k =$ **k**. Since $k = \log_2 n$, we get $\mathbf{A(n) = \log_2 n \in \Theta(\log n)}$.

Fibonacci (slides show only the code)

```
Fib(n): if n = 0 return 0
        if n = 1 return 1
        else return Fib(n−1) + Fib(n−2)
```

Its recurrence is $A(n) = A(n-1) + A(n-2) + 1$. The standard result is that it grows **exponentially**, so this simple recursion is very slow.

9. Binary search

Like guessing a number between 1 and 100: always guess the middle, and the answer tells you which half to throw away. It works only on a **sorted list**.

1. Look at the middle element.
2. If it equals the target, stop.
3. If the target is smaller, search the left half. If larger, search the right half.
4. Repeat on the half that remains.

Slide example: list 1 to 16, target 11. Probes: 8 → 12 → 10 → **11** found.

Why it is fast: every check removes about half the elements. With $N = 2^k - 1$ elements, the sizes go 2^k-1, $2^{k-1}-1$, $2^{k-2}-1$, ..., down to 1. The worst case is finding the target on the last pass or not finding it at all, which takes k passes:

$$k = \log_2(N + 1) \Rightarrow C_{worst}(n) \in \Theta(\log n)$$

For 1,000,000 items, that is only about 20 checks, against up to 1,000,000 for sequential search.

10. Last-minute revision

Results table (memorise)

Algorithm	Basic operation	Count	Class
Sequential search	Key comparison	best 1 · worst n · avg $p(n+1)/2 + n(1-p)$	$O(n)$
Maximum element	A[i] > maxval	$n - 1$	$\Theta(n)$
Element uniqueness (worst)	A[i] = A[j]	$n(n-1)/2$	$\Theta(n^2)$
Matrix multiplication	Multiplication	n^3	$\Theta(n^3)$
Factorial (recursive)	Multiplication	n	$\Theta(n)$
Tower of Hanoi	Disc move	$2^n - 1$	$\Theta(2^n)$
BinRec	Addition	$\log_2 n$	$\Theta(\log n)$
Binary search (worst)	Comparison	$\log_2(n+1)$	$\Theta(\log n)$

Questions likely to come

1. Define an algorithm and write its characteristics.
2. Explain the steps of algorithm design and analysis.
3. Euclid's algorithm for gcd; compare with consecutive integer checking.
4. Explain the problem types. What are in-place and stable sorting?
5. Linear vs non-linear data structures; set, bag, dictionary.
6. Explain input size, basic operation, order of growth, best/worst/average case.
7. Analyse sequential search for all three cases.
8. Define O, Ω, Θ with graphs; prove $100n + 5 \in O(n^2)$ and $\frac{1}{2}n(n-1) \in \Theta(n^2)$.
9. State and prove: $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$.
10. Compare growth using limits and L'Hôpital's rule ($\log n$ vs $\sqrt{n}$).
11. Plan for non-recursive analysis; analyse maximum element, uniqueness, matrix multiplication.
12. Plan for recursive analysis; solve factorial and Tower of Hanoi.
13. Analyse BinRec; explain the smoothness rule.
14. Explain binary search and its worst-case analysis.
15. Short notes: power set, equivalence relation, bijective function, dot product, space vs time complexity.

Traps and slide mistakes

- The slides say Tower of Hanoi needs "(n − 1)" moves. That is wrong. The correct answer is **$2^n - 1$**.
- The factorial code on slide 79 is a loop, but the recurrence $M(n) = M(n-1) + 1$ is for the **recursive** version F(n).
- Average case is **not** the mean of best and worst.
- Big-O is only an upper bound. Use Θ when the bounds match.
- In every proof, **state c and n_0**. That is where the marks are.